X-OmniClaw

当前版本 | 支持安卓 | 开源许可证 | 技术论文

作为一个集语音、视觉与动作于一体的 **安卓原生智能体**，**X-OmniClaw** 的定位是「可执行的图形界面引擎」。它不依赖虚拟环境，而是直接在真机上沿「感知-执行-优化」的逻辑循环推进。

Omni 意味着全方位的感知能力——系统不仅关注屏幕内的像素，还关注屏幕外的物理上下文；而 **X-** 则赋予系统跨模态的整合能力。这种架构将碎片化的数据整合为统一的意志，确保智能体能够像人类一样，在真实的移动端生态中自主、可靠地完成任务闭环。

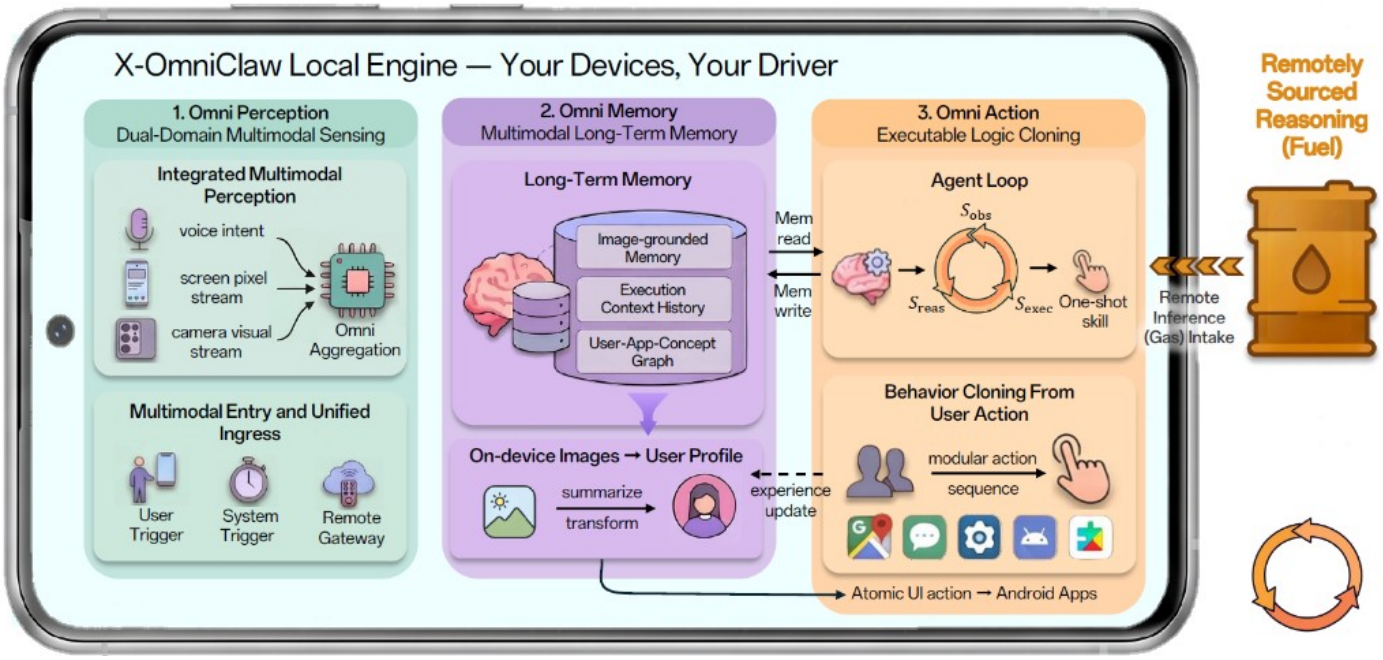
本文（中文） · English 英文

🧭 概览 | 💡 核心特性 | 🎬 使用场景与演示 | 🧩 技能与工具 | 👹 模型 | 🚀 快速开始 | 🛠️ 源码构建 | 🧡 贡献指南 | 📄 许可证 | 🙏 致谢

📣 动态

- **2026-04-23** 📊 多轮调用可观测性升级：智能体多轮 **令牌用量** 开始累计并展示到主界面、飞书与 Discord 渠道及**网关**，同时补齐 OpenRouter 模型列表拉取与配置链路。
- **2026-04-22** 🧠 执行策略收敛：确立「纯文本可本地直返、凡设备操作统一走智能体」的主路径，并增强跨包 ref 安全重绑定与错误定位日志。
- **2026-04-20** 🧵 多会话并行落地：会话级独立智能体主循环、跨会话运行态隔离与精确停止链路上线，显著提升长任务稳定性。
- **2026-03-31** ⏰ 定时自动化可用：支持按间隔、工作日或周计划执行任务，覆盖息屏与亮屏场景。
- **2026-03-25** 🎙️ 语音视觉主链路成型：本地语音视觉闭环（录音、视觉帧、决策与执行）打通，语音与文本进入统一执行核心。
- **2026-03-14** 🛠️ 核心运行时重构：统一设备工具落地（快照、操作、打开应用、截图等），并对齐 OpenClaw 关键运行时能力。

🧭 概览



X-OmniClaw 定位为面向智能手机的 **全模态移动 Agent**：不依赖虚拟环境，在真机上沿 **感知** → **策略** → **执行** → **验收** 闭环推进。下文用表格归纳其 **执行型方法论**、**四层架构** 与 **三大全模态支柱**。

1. 核心方法论：从「对话」转向「执行」

X-OmniClaw 遵循 **循环递归推理** 模型，确保在复杂移动端 GUI 环境下的任务达成。

环节	标识 (Action)	描述
观察	**Observe**	通过系统快照、实时截图或无障碍树（Accessibility Tree）获取当前界面状态。
思考	**Think**	结合用户目标、当前上下文与历史动作记录，由 LLM/VLM 做出决策。
执行	**Act**	调度底层驱动执行点击、滑动、文本输入或应用切换等原子动作。
验证	**Verify**	动作完成后重新捕获界面，对比预期结果与实际状态。
循环	**Loop**	若任务未完成，将新状态作为下一轮输入；若已达成，则结束任务。

2. 系统技术架构：四层闭环驱动

系统采用模块化设计，通过「感知 → 策略 → 执行 → 验收」实现全链路闭环。

架构层	核心组件	职责定义
感知层	**Multi-modal Input**	融合语音识别（ASR）、截图/录屏帧、无障碍树信息，构建统一感知上下文。
策略层	**Agent Logic**	智能体主循环中心，负责任务拆解，通过 Kotlin 桥接进行逻辑分发。
执行层	**Device Scheduler**	硬件/系统调度器，执行快照抓取、UI 模拟操作及应用生命周期管理。
验收层	**Success Monitor**	执行后的状态回溯与循环检测，判断逻辑是否偏移或任务是否终结。

3. X-OmniClaw 核心优化：三大全模态支柱

相较传统 Agent，X-OmniClaw 在 **感知深度、记忆广度与动作鲁棒性** 上做专项强化；三者协同支撑真实应用任务。

特性支柱	核心优化点	技术实现
Omni Perception（全感知）	多源融合与意图分解	融合界面元数据与真实世界视觉（相机），在决策前将原始输入蒸馏为结构化意图。
Omni Memory（全记忆）	跨应用任务连续性	结合多模态工作记忆与个人长期记忆，支持从个人相册等私域数据中获取语义线索。
Omni Action（全行动）	异构交互与技能抽象	基于视觉 Grounding 的稳健交互，支持行为克隆（BC）与轨迹回放，实现复杂操作的复用。

💡 核心特性

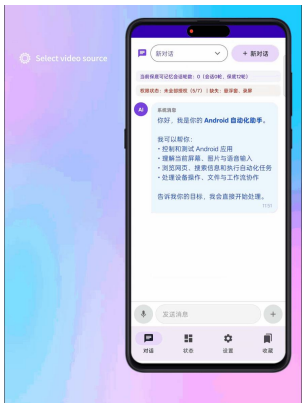
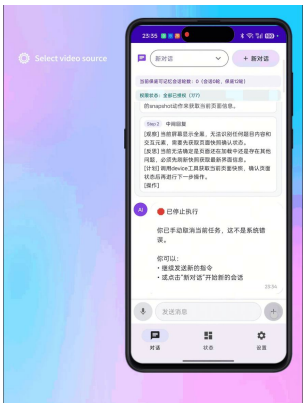
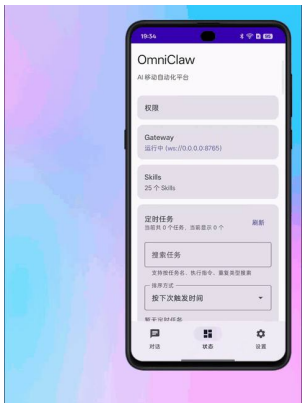
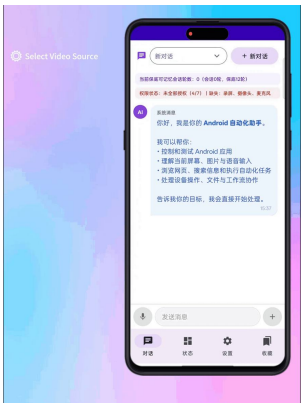
- **端侧可执行智能体闭环**：多轮任务不是一次性问答，而是带预算与循环检测的执行循环，会在失败时收敛并继续推进。
- **过程与成本可观测**：执行过程可实时输出（步骤、思考、工具调用与结果等），并对大模型用量做累计，便于界面侧展示。
- **统一设备工具入口**：界面理解、点击输入、打开应用、截图与剪贴板等操作走同一套设备工具，并内置稳定性与误触保护。
- **视觉兜底与双轨决策**：优先结构化理解，同时在复杂页面或弱结构场景下可用视觉兜底降低误操作。
- **语音到动作的一体链路**：语音识别后结合屏幕画面理解意图，并能把「按下说话那一刻」的画面对齐到决策输入。
- **相册与媒体向的落地能力**：支持把相册内容纳入「可检索、可总结、可执行」的工作流（例如按问题查图、做记忆整理，并串到剪映类一键成片等场景）。
- **深度链接与可复现流程（在能力可用时）**：可维护深度链接与收藏，把长路径或反复操作压缩成可复用的一键入口，适合「录一次、下次一句话直达」。
- **多会话并行与可控停止**：支持并行会话互不干扰，并可按会话中断任务。

🎬 使用场景与演示

以下为典型指令与执行要点；与下方动图演示一一对应，动图与本仓库 **videos/** 目录一致。

🎥 动图演示

📷 演示 1.1 — 相机感知的操作	📺 演示 1.2 — 投屏或屏幕伴侣	✂️ 演示 2 — 记忆驱动的一键成片	🤏 演示 3 — 行为克隆
用户指令 「这瓶水在淘宝上卖多少钱」	**用户指令** 「开始做题吧。」	**用户指令** 「帮我找到与鹦鹉主题相关的照片并一键成片。」	**用户指令** 「打开美团秒杀」
执行特征 • 用**相机画面**识别水瓶/包装	**执行特征** • 基于**当前屏幕/投屏**锁定题	**执行特征** • 在**相册**侧按「鹦鹉」等主	**执行特征** • 将「打开美团秒杀」解析

📷 演示 1.1 — 相机感知的操作	📺 演示 1.2 — 投屏或屏幕伴侣	✂️ 演示 2 — 记忆驱动的一键成片	📦 演示 3 — 行为克隆
与文字，对齐「这瓶水」所指对象 • **跳转淘宝**：搜索或图搜进入候选商品与详情页 • **多轮核对**价格、规格与店铺信息后，汇总「卖多少钱」的结论	目区域与题型（选择、填空等） • 按题目顺序**作答或点选**，每步后观察界面是否进入下一题 • **未完成则循环**：错题或卡住时可换策略继续，直至本批题目处理完	题**检索、筛选**相关照片作为成片素材 • **串联剪辑类流程**：选图 → 选题或模板 → 触发一键成片步骤 • **中途验收**：成片预览或导出节点可再调整再继续	为**应用内目标路径**（唤起美团客户端） • **导航至秒杀/活动入口**（首页入口、搜索或已保存路径） • 依托**行为克隆/轨迹复现**：对已记录步骤顺序回放或等价点击，直达目标页
相机实物 → 淘宝比价询价闭环。	**读屏做题**：投屏/同屏上下文下逐步作答。	**相册主题召回 → 一键成片**串联。	**直达美团秒杀**（应用唤起 + 路径导航 / 轨迹复现）。
			

🔧 技能与工具

执行任务时，先有按需加载的**技能扩展**，再通过底层**工具**（尤其设备工具 device）把动作落到真实界面；而不是只做一次性文本生成。

📁 随包技能（共 10 个）

下列技能随安装包内置，源码目录为客户端工程 app/src/main/assets/skills/（每个目录含 SKILL.md）。

类别	技能标识
搜索与应用	app-search, taobao-search
相册与媒体	gallery-qa, gallery-memory, capcut-theme-video, clipboard-to-shortcut
配置管理	model-config, channel-config
技能管理	skill-creator
自动化	scheduled-automation

🧪 各技能示例指令（可直接对智能体说）

技能标识	示例指令
app-search	「去小红书搜北京旅游攻略发给我。」

技能标识	示例指令
taobao-search	「淘宝搜『男士轻薄防晒衣』，按价格区间和销量给我推荐 3 个。」
gallery-qa	「我今天拍了什么照片？按时间顺序简单说一下。」
gallery-memory	「同步一下相册记忆并更新用户画像，先扫最近 20 张。」
clipboard-to-shortcut	「把剪贴板里的链接做成一个技能，名字叫『淘宝好物直达』。」
channel-config	「把飞书渠道配置好并启用，应用编号是 xxx，密钥是 xxx。」
model-config	「新增一个兼容开放式接口的提供商，接口地址是 xxx，默认模型设成 xxx。」
scheduled-automation	「每周三早上 10 点打开小红书搜索人工智能新闻并总结给我。」
capcut-theme-video	「把今天拍的风景图做成剪映一键成片，素材放到 A_latest 后开始。」
skill-creator	「帮我把刚刚执行成功的方法总结成新的技能，并生成对应的 SKILL.md。」

模型

> 本项目不捆绑单一模型，而是「提供商 + 模型引用」的可配置架构。
> 你可以按场景选择不同模型，语音视觉链路与文本链路共享同一套提供商体系。
>
> ⚠ **重要**：本项目依赖**视觉语言模型**做截图与界面理解，因此提供商与主模型必须支持**图像输入（多模态）**；
> 仅用文本模型会导致视觉相关任务（如图形界面理解、语音视觉任务）明显降级或失败。

支持的提供商与预置模型（代码内置）

提供商 ID	示例预置模型 ID	示例界面名称
openrouter	z-ai/glm-4.5-air:free	GLM 4.5 Air (免费)
anthropic	claude-opus-4	Claude Opus 4
openai	gpt-4.1	GPT-4.1
google	gemini-2.5-pro	Gemini 2.5 Pro
ollama	—	无固定预置；依赖 /api/tags 自动发现
moonshot	kimi-k2.5	Kimi K2.5
xiaomi	mimo-v2-flash	MiMo V2 Flash (免费，262K)
minimax	MiniMax-M2.5	MiniMax M2.5
bot-adaptor	doubao-seed-2.0-lite	豆包 Seed 2.0 Lite

配置示例

```
{
  "models": {
    "providers": {
      "openrouter": {
        "baseUrl": "https://openrouter.ai/api/v1",
```

```
      "apiKey": "sk-or-v1-xxx"
    }
   }
  },
  "agents": {
   "defaults": {
    "model": {
     "primary": "openrouter/z-ai/glm-4.5-air:free"
    }
   }
  }
}
```

🎙️ 语音与视觉参数

视觉与语音参数位于配置中的 vision 段，包括帧率、画质、语音转写服务等：

> 🔑 语音转写使用**单独的接口密钥**，不复用 models.providers 中的模型密钥。
> 可前往 **硅基流动官网** 申请密钥；配置项 sttUrl 请以硅基流动控制台或文档给出的**语音转写接口地址**为准（下方仅为常见对接示例）。

```
{
  "vision": {
    "fps": 1,
    "quality": 80,
    "sttUrl": "https://api.siliconflow.cn/v1/audio/transcriptions",
    "sttApiKey": "sk-xxx"
  }
}
```

🚀 快速开始

📥 1）下载与安装

从发行版页面获取安装包：
https://github.com/OPPO-Mente-Lab/X-OmniClaw/releases/latest

⚙️ 2）首次配置

1. 打开应用
2. 配置支持**视觉语言多模态**的提供商与接口密钥（必须支持图像输入）
3. 授予关键权限（共 **7** 项，与应用内权限检测一致）：**无障碍服务**，**悬浮窗**，**录屏**，**相册（照片与视频）**，**全部文件访问**，**摄像头**，**麦克风**
4. 根据需要配置渠道（飞书与 Discord）

📄 3）配置文件位置

```
/sdcard/.xomniclaw/xomniclaw.json
```

🛠️ 源码构建

📋 环境要求

- 不低于 JDK 17
- 安卓软件开发工具包
- Gradle 封装脚本（推荐）

🔨 克隆与构建（示例）

```
git clone https://github.com/OPPO-Mente-Lab/X-OmniClaw.git
cd X-OmniClaw
./gradlew :app:assembleDebug
```

系统自带 PowerShell 示例：

```
$env:JAVA_HOME = "D:\path\to\jdk-17"
$env:ANDROID_HOME = "D:\path\to\android-sdk"
Set-Location "D:\path\to\X-OmniClaw"
.\gradlew.bat :app:assembleDebug
```

输出安装包（示例路径）：

```
releases/X-OmniClaw-v<version>-debug.apk
```

构建观察者组件：

```
./gradlew :extensions:observer:assembleDebug
```

🤝 贡献指南

欢迎提交合并请求与议题，欢迎文档改进与场景反馈。

📄 许可证

本项目采用 Apache License 2.0。

许可要点：

- ✅ 允许商用
- ✅ 允许修改与再分发
- ✅ 包含专利授权相关条款
- ⚠️ 须保留版权声明
- ⚠️ 修改内容须予以说明

🙏 致谢

感谢所有为本项目做出贡献的开发者和研究者。

如果这个项目对你有帮助，请给个 Star 支持开源！